CSCI-4208: Developing Advanced Web Applications

Week 11: Lecture 19

HTTP Server

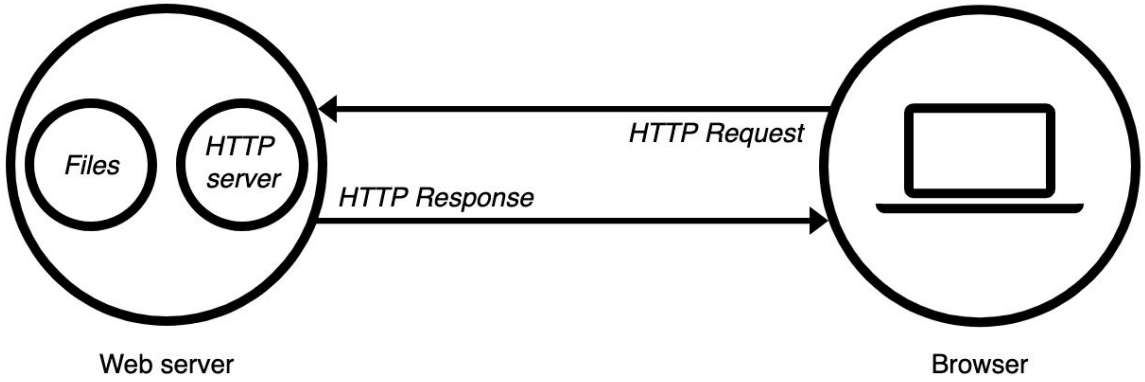
HTTP Servers: Overview

1. Introduction
2. HTTP Server Responsibilities
3. Static vs Dynamic Servers
4. Request/Response Objects
5. HTTP Routes & Response Handlers
6. HTTP Status Codes
7. Mime Types
8. HTTP Headers
9. HTTP Sessions
10. HTTP Authentication
11. HTTP CORS

1. Introduction

What is a Web Server? What is a HTTP Server?

A HTTP server is an application that listens for client HTTP requests and sends a response along. A web server is an HTTP server with additional applications or files necessary for it to provide a response.



2. Web Server Responsibilities

- Listens for HTTP Request
- Provide Response & Close Connection
- Maintains Private Data (User Accounts, API Keys)
- Defines the Source of Truth (Data)

3. Static vs Dynamic Servers

To publish a website, you need either a static or a dynamic web server.

Static web server

consists of a HTTP server (software). Its "static" because the server sends its hosted files as-is to your browser. Thus it only responds to HTTP GET requests to serve back files. No server-side logic or code!

Dynamic web server

consists of a static web server plus extra software, most commonly an application server and a database. We call it "dynamic" because the application server updates the hosted files before sending content to your browser via the HTTP server. Dynamic servers may respond to different types of HTTP requests. There is server-side logic and code.

4. Request/Response Objects

- **Request**
  - Request maintains meta information about the incoming request
  - Request contains a body or string query with data from client
  - Request may also include session/token objects & user info to identify a client

- **Response**
  - Response object maintains the information to send back to the client
  - Response object has methods for sending data and ending the connection

5. HTTP Routes & Response Handlers

Routes:

Routing refers to how an application's endpoints (URIs) respond to client requests. A HTTP server listens:

- To a specified URL path
- For a specified HTTP method *(get, post, put, delete)*

Response Handler:

The server function that handles the request and sends back a response to the client.

6. Status Codes

HTTP servers include a status code in their response to indicate whether a specific HTTP request has been successfully completed. Responses are grouped in five classes:

- Informational responses (100–199),
- Successful responses (200–299),
- Redirects responses (300–399),
- Client error responses(400–499),
- Server error responses (500–599).

https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

6. Status Codes - Common Codes

Status code 200 – OK: The HTTP status code 200 shows that the request was successfully carried out.

Status code 301 – Moved Permanently: means that the data requested from the client cannot be found under the given address since it has been moved permanently.

Status code 302 – Moved Temporarily: informs that the requested data has temporarily been moved.

Status code 403 – Forbidden: the requested data is access-protected and that the request cannot be performed due to the client not having authority.

Status code 404 – Not Found: means that the requested website information was not found on the server.

Status code 500 – Internal Server Error: a collection status code for unexpected server errors.

Status code 503 – Service Unavailable: means that the relevant web server, which should deliver the requested information, is overloaded.

7. Mime Types

MIME Types: A media type (also known as a Multipurpose Internet Mail Extensions or MIME type) is a standard that indicates the nature and format of a document, file, or assortment of bytes.

Browsers use the MIME type, not the file extension, to determine how to process a URL, so it's important that web servers send the correct MIME type in the response `Content-Type` header.

https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP/MIME_types

7. Mime Types

Application Type		
Type/Subtype	**Extensions**	**Description**
application/msword	.doc	Microsoft Word file.
application/octet-stream	.bin, .exe	Any binary file, shows file download/save dialog box.
application/pdf	.pdf	Adobe Acrobat file.
application/postscript	.ps, .ai, .eps	PostScript files.
application/rtf	.rtf	Rich Text Format.
application/x-gtar	.gtar	Compressed Linux file.
application/x-gzip	.gz	Compressed Linux file.
application/x-java-archive	.jar	Java .jar file.
application/x-java-serialized-object	.ser	Java .ser file.
application/x-java-vm	.class	Java .class file.
application/x-tar	.tar	Compressed Linux file.
application/zip	.zip	ZIP compressed file.

Audio Type		
Type/Subtype	**Extensions**	**Description**
audio/x-aiff	.aiff	Apple sound file.
audio/basic	.ua	Basic 8-bit ULAW file.
audio/x-midi	.mid, .midi	MIDI sound file.
audio/x-wav	.wav	WAV sound file.

7. Mime Types

Image Type		
Type/Subtype	**Extensions**	**Description**
image/bmp	.bmp	Bitmap image file.
image/gif	.gif	Graphics Interchange Format.
image/jpeg	.jpeg, .jpg, .jpe	JPEG is another image compression format.
image/tiff	.tiff, .tif	Tag Image File Format.
image/x-xbitmap	.xbm	X bitmap format.

Multipart Type		
Type/Subtype	**Extensions**	**Description**
multipart/x-gzip	.gzip	Compressed Linux file.
multipart/x-zip	.zip	ZIP compressed file.

Text Type		
Type/Subtype	**Extensions**	**Description**
text/html	.htm, .html	HTML file.
text/plain	.txt	Plain text file.
text/richtext	.rtf, .rtx	Rich text format.

Video Type		
Type/Subtype	**Extensions**	**Description**
video/mpeg	.mpg, .mpeg, .mpe	MPEG compressed video.
video/vnd.vivo	.viv, .vivo	VIVO video codec.
video/quicktime	.qt, .mov	Apple's QuickTime format.
video/x-msvideo	.avi	Microsoft's video format.

8. HTTP Headers

There are several categories of HTTP Headers to relay information about the request or response. The following slides details the various types of headers along with a description

8. HTTP Headers

Authentication

`WWW-Authenticate`

Defines the authentication method that should be used to access a resource.

`Authorization`

Contains the credentials to authenticate a user-agent with a server.

`Proxy-Authenticate`

Defines the authentication method that should be used to access a resource behind a proxy server.

`Proxy-Authorization`

Contains the credentials to authenticate a user agent with a proxy server.

8. HTTP Headers

Caching

`Age`
The time, in seconds, that the object has been in a proxy cache.

`Cache-Control`
Directives for caching mechanisms in both requests and responses.

`Clear-Site-Data`
Clears browsing data (e.g. cookies, storage, cache) associated with the requesting website.

`Expires`
The date/time after which the response is considered stale.

`Pragma`
Implementation-specific header that may have various effects anywhere along the request-response chain. Used for backwards compatibility with HTTP/1.0 caches where the `Cache-Control` header is not yet present.

`Warning`
General warning information about possible problems.

8. HTTP Headers

Connection management

`Connection`

Controls whether the network connection stays open after the current transaction finishes.

`Keep-Alive`

Controls how long a persistent connection should stay open.

8. HTTP Headers

Content negotiation

`Accept`

Informs the server about the types of data that can be sent back.

`Accept-Charset`

Which character encodings the client understands.

`Accept-Encoding`

The encoding algorithm, usually a compression algorithm, that can be used on the resource sent back.

`Accept-Language`

Informs the server about the human language the server is expected to send back. This is a hint and is not necessarily under the full control of the user: the server should always pay attention not to override an explicit user choice (like selecting a language from a dropdown).

8. HTTP Headers

Cookies

`Cookie`

Contains stored HTTP cookies previously sent by the server with the `Set-Cookie` header.

`Set-Cookie`

Send cookies from the server to the user-agent.

`Cookie2`

Contains an HTTP cookie previously sent by the server with the `Set-Cookie2` header, but has been **obsoleted**. Use `Cookie` instead.

`Set-Cookie2`

Sends cookies from the server to the user-agent, but has been **obsoleted**. Use `Set-Cookie` instead.

8. HTTP Headers

Message body information

`Content-Length`
The size of the resource, in decimal number of bytes.

`Content-Type`
Indicates the media type of the resource.

`Content-Encoding`
Used to specify the compression algorithm.

`Content-Language`
Describes the human language(s) intended for the audience, so that it allows a user to differentiate according to the users' own preferred language.

`Content-Location`
Indicates an alternate location for the returned data.

8. HTTP Headers

Redirects

`Location`

Indicates the URL to redirect a page to.

8. HTTP Headers

Request context

`From`
Contains an Internet email address for a human user who controls the requesting user agent.

`Host`
Specifies the domain name of the server (for virtual hosting), and (optionally) the TCP port number on which the server is listening.

`Referer`
The address of the previous web page from which a link to the currently requested page was followed.

`Referrer-Policy`
Governs which referrer information sent in the `Referer` header should be included with requests made.

`User-Agent`
Contains a characteristic string that allows the network protocol peers to identify the application type, operating system, software vendor or software version of the requesting software user agent. See also the Firefox user agent string reference.

8. HTTP Headers

Response context

`Allow`

Lists the set of HTTP request methods supported by a resource.

`Server`

Contains information about the software used by the origin server to handle the request.

9. HTTP: Session (HTTP Server)

"Session" is the term used to refer to a user's time browsing a web site. It's meant to represent the time between their first arrival at a page in the site until the time they stop using the site. In practice, it's impossible to know when the user is done with the site. In most servers there's a timeout that automatically ends a session unless another page is requested by the same user.

The first time a user connects some kind of session ID is created (how it's done depends on the web server software and the type of authentication/login you're using on the site). Like cookies, this usually doesn't get sent in the URL anymore because it's a security problem. Instead it's stored along with a bunch of other stuff that collectively is also referred to as the session. Session variables are like cookies - they're name-value pairs sent along with a request for a page, and returned with the page from the server - but their names are defined in a web standard.

Some session variables are passed as HTTP headers. They're passed back and forth behind the scenes of every page browse so they don't show up in the browser and tell everybody something that may be private. Among them are the USER_AGENT, or type of browser requesting the page, the REFERRER or the page that linked to the page being requested, etc. Some web server software adds their own headers or transfer additional session data specific to the server software. But the standard ones are pretty well documented.

10. HTTP: Authentication - (HTTP Server)

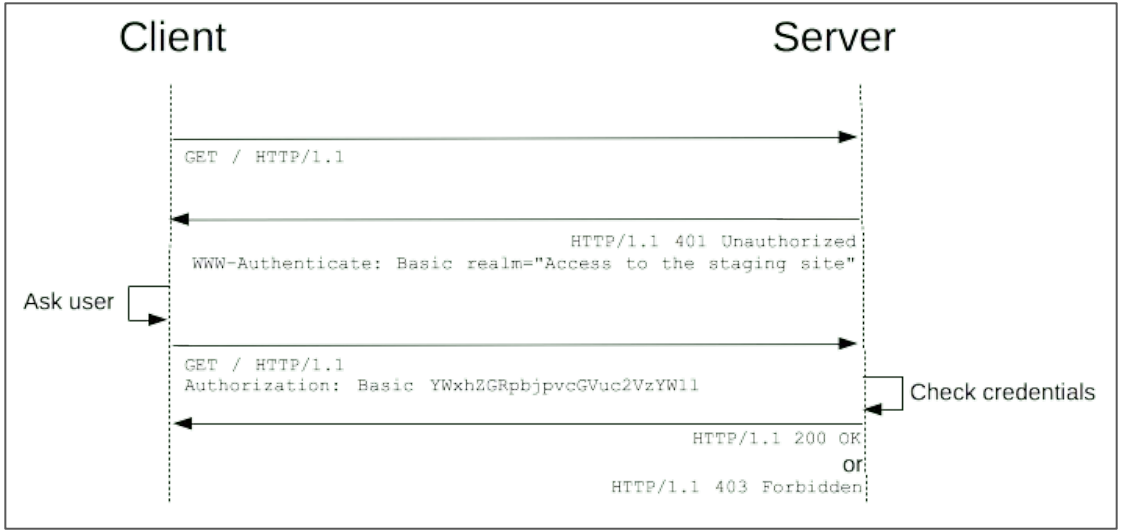
HTTP provides a general framework for access control and authentication. HTTP authentication framework, which can be used by a server to challenge a client request, and by a client to provide authentication information

The challenge and response flow works like this:

1. The server responds to a client with a `401` (Unauthorized) response status and provides information on how to authorize with a `WWW-Authenticate` response header containing at least one challenge.

2. A client that wants to authenticate itself with the server can then do so by including an `Authorization` request header with the credentials.

3. Usually a client will present a password prompt to the user and will then issue the request including the correct `Authorization` header.



11. HTTP CORS

Cross Origin Resource Sharing (CORS)

To prevent snooping and other privacy invasions, Web browsers enforce strict separation between Web sites. Only pages from the same origin can access all the information of a Web page. Though such constraint is a burden to the server, HTTP headers can relax this strict separation on the server side, allowing a document to become a patchwork of information sourced from different domains; there could even be security-related reasons to do so.

Web Servers may relax the origin constraint

The End.

Coming Soon... to a Moodle near you!

Labs	HTTP Server in Node + NPM
Homework	REST Client App